

Project 4: Text Processing Documentation and Analysis

Overview

In this assignment, you will enhance a provided text processing notebook by adding comprehensive documentation and conducting a comparative analysis of word frequency results. You will use **Google Colab's built-in Gemini "Explain Code" feature** to help you understand functions before writing your own docstrings — practicing a skill that is essential in professional software development. You can also use Gemini to help write the docstrings if you want to.

This project builds on your Lab 4 work with text processing and introduces you to standard documentation practices used in collaborative software environments.

Objectives

- Use Gemini's Explain Code feature in Google Colab to understand and document existing code
- Write (or generate with Gemini) effective docstrings for all functions in the provided notebook
- Understand how text preprocessing with spaCy affects word frequency analysis
- Compare results from basic and advanced text processing approaches
- Write a clear, thoughtful comparative analysis

Getting Started: Setting Up Your Notebook

The project notebook is shared with you via a link on Canvas.

To make your own editable copy:

1. Open the shared notebook link
2. Go to **File → Save a copy in Drive**
3. The copy will open automatically in your Google Drive — this is the version you will work in and submit
4. Rename it: `Project4.ipynb`

Do not edit the original shared notebook. You must work in your own copy saved to Drive.

What Are Docstrings?

A **docstring** (documentation string) is a special string that appears as the first statement inside a function definition. In Python, docstrings are written using triple quotes ("""") and serve as built-in documentation for your code.

Why Docstrings Matter

- They make code easier to understand for others — and for your future self
- They appear when users call `help()` on a function
- They are used by documentation generation tools
- They are a professional standard in software development
- They help AI coding assistants better understand your intentions

Example of a Well-Written Docstring

```
def calculate_average(numbers):  
    """  
    Calculate the arithmetic mean of a list of numbers.  
  
    This function takes a list of numeric values and returns their average.  
    Empty lists will return 0 to avoid division by zero errors.  
  
    Parameters:  
        numbers (list): A list of int or float values to average  
  
    Returns:  
        float: The arithmetic mean of the input numbers, or 0 if the list is  
empty  
  
    Example:  
    >>> calculate_average([1, 2, 3, 4, 5])  
    3.0  
    >>> calculate_average([10, 20])  
    15.0  
    """  
    if len(numbers) == 0:  
        return 0  
    return sum(numbers) / len(numbers)
```

What a Good Docstring Includes

Section	Description
Brief description	One concise sentence describing what the function does
Detailed explanation <i>(optional)</i>	Extra context if the function is complex
Parameters	Each parameter's name, type, and

Section	Description
	description
Returns	The return type and what it represents
Example (<i>optional but helpful</i>)	A short usage example

Part 1: Use Gemini’s “Explain Code” Feature

Google Colab has a built-in AI assistant powered by Gemini. You will use this feature to help you understand each function — and you may also use it to help **generate your docstrings**.

How to Use Explain Code

1. Click on a code cell containing a function
2. Look for the  **Gemini** button or the sparkle icon in the top-left corner of the cell (or in the toolbar)
3. Select **“Explain code”**
4. Read Gemini’s explanation carefully

Using Gemini to Generate Docstrings

You may also ask Gemini to write a docstring for you. To do this:

1. Click the  **Gemini** button on a code cell
2. In the chat input, type something like: *“Write a docstring for this function”*
3. Review the generated docstring carefully before accepting it

Whether you write docstrings yourself or use Gemini to generate them, you are responsible for ensuring they are **accurate and complete**. Always verify that the docstring matches what the function actually does.

Part 2: Add Docstrings to All Functions

After using Gemini to understand each function, write a complete, professional docstring for **all seven functions** in the notebook:

1. `fetch_text(raw_url)`
2. `print_text_stats(text)`
3. `get_word_counts(text)`
4. `print_top_10_frequent_words(text)`
5. `word_tokenization_normalization(text)`
6. `word_count(word_list)`
7. `print_top_15_frequent_words(word_counts)`

Requirements for Each Docstring

- A clear description of what the function does
- A **Parameters** section listing each parameter with its type and description
- A **Returns** section describing the return value and its type (if applicable)
- Any important notes about edge cases or behavior

Whether written by you or generated by Gemini, every docstring must be accurate. Read each one carefully and correct anything that does not match the function's actual behavior.

Part 3: Comparative Analysis

After adding docstrings and running all the code cells, write a **comparative analysis** at the end of the notebook in a designated text cell (Markdown cell).

Analysis Questions to Address

Your analysis should answer all of the following:

- 1. Top 10 vs. Top 15 Comparison** - the top 10 most frequent words from the basic `get_word_counts()` function vs the top 15 most frequent words after using spaCy's `word_tokenization_normalization()` - How do these two lists differ?
- 2. Impact of Text Preprocessing** - Why are the results different between the two approaches? What does spaCy's preprocessing remove or change? Think about: stop words, punctuation, lemmatization, and minimum word length Which approach gives more meaningful results for understanding the text's content, and why?
- 3. Insights About the Text** - Based on the top 15 words from the cleaned output, what can you infer about the themes or topics in this *Pride and Prejudice* excerpt? How does removing common words like “the”, “and”, and “to” help reveal the text's actual content?

Length and Format

- Write **150–250 words**
- Use complete sentences and clear paragraphs
- Be specific — reference actual words from the output, not just general statements

Submission Checklist

Before submitting, confirm:

- ☐ You saved a copy of the notebook to your own Google Drive
- ☐ All 7 functions have complete docstrings

- ☐ All code cells run without errors
- ☐ Your comparative analysis is written in the designated Markdown cell at the end
- ☐ Your analysis is 150–250 words and addresses all three question areas
- ☐ You downloaded the notebook as a .ipynb file: **File → Download → Download .ipynb**
- ☐ You uploaded the .ipynb file to the Project 4 assignment on Canvas

Evaluation Criteria (5 Points Total)

1. Docstring Quality — 2.5 points

Score	Description
2.5 — Excellent	All 7 functions have complete, accurate docstrings with correct parameter types, return descriptions, and clear explanations. Follows professional formatting.
2.0 — Good	All functions documented, but some docstrings lack detail or have minor formatting issues.
1.5 — Satisfactory	Most functions documented, but several are missing parameters or return descriptions.
1.0 — Needs Improvement	Documentation incomplete or contains significant errors. Multiple functions missing docstrings.
0–0.5 — Insufficient	Minimal or no documentation provided.

2. Comparative Analysis — 1.5 points

Score	Description
1.5 — Excellent	Thorough, insightful analysis (150–250 words) explaining differences between the two approaches. References specific words from the output and demonstrates understanding of preprocessing.
1.2 — Good	Addresses all questions with solid understanding, though may lack depth or specific examples.
0.9 — Satisfactory	Addresses most questions but analysis is surface-level or misses key insights.
0.6 — Needs Improvement	Incomplete, too brief, or shows misunderstanding of preprocessing

Score	Description
0–0.3 — Insufficient	concepts. Minimal or no analysis provided.
3. Code Functionality — 1 point	
Score	Description
1.0 — Excellent	All cells run without errors. Both word frequency results display correctly.
0.8 — Good	Code runs with minor warnings but produces correct results.
0.6 — Satisfactory	Code runs but has some functionality problems or incomplete output.
0.4 — Needs Improvement	Significant errors or missing expected output.
0 — Insufficient	Code does not run or notebook was not submitted.

Academic Integrity

- You are encouraged to use Gemini’s Explain Code feature and to have Gemini generate docstring drafts
 - You **must** review and verify all AI-generated content — do not accept it without reading it
 - Your comparative analysis must be written in your own words
 - Collaboration on understanding the material is encouraged, but your submitted work must reflect your own understanding
-

Why This Matters

Documentation is not optional in professional software development — it is essential. Developers spend 50–70% of their time reading and understanding existing code rather than writing new code. Good documentation saves time, prevents errors, and enables collaboration.

Understanding how text preprocessing affects analysis is equally important for data science, natural language processing, and any work involving text mining. The skills you practice in this project — reading code critically, documenting clearly, and comparing analytical approaches — are directly applicable to internships, research positions, and professional development roles.

Good code is not just code that works. It is code that others — and future you — can understand and maintain.